

Sampling

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Basic Sampling Algorithms	2
2.1	Inverse Transform Sampling	2
2.2	Rejection Sampling	3
2.3	Adaptive Rejection Sampling	4
2.4	Importance Sampling	5
2.5	Sampling-importance-resampling	5
3	Markov Chain Monte Carlo	7
3.1	Introduction	7
3.2	Markov Chain	7
3.3	Metropolis-Hastings Algorithm	8
3.4	Gibbs Sampling	8
3.5	Ancestral Sampling	9
4	Langevin Sampling	9

1 Introduction

In this document, we discuss about sampling methods mainly used in deep learning. We use sampling methods to handle imbalanced dataset, to create a generated data from generative model, to calculate marginal distribution or to estimate the uncertainty of prediction. A lot more usage can be found elsewhere.

2 Basic Sampling Algorithms

2.1 Inverse Transform Sampling

Algorithm 1: Inverse transform sampling algorithm

Input : desired distribution $p(\cdot)$

Output: $z \sim p(z)$

```
1  $h(x) \leftarrow$  the inverse of the indefinite integral of  $p(x)$ .  
2  $u \sim \mathcal{U}(0, 1)$ .  
3  $z \leftarrow h(x)$   
4 return  $z$ 
```

Proof. Suppose $Y = g(Z) \sim p(x)$ where $Z \sim \mathcal{U}(0, 1)$. We want to obtain $g(\cdot)$. Then,

$$\begin{aligned} \int_{-\infty}^y p(t) dt &= \mathbb{P}(Y \leq y) \\ &= \mathbb{P}(g(Z) \leq y) \\ &= \mathbb{P}(z \leq g^{-1}(y)) \\ &= g^{-1}(y) \end{aligned}$$

Therefore $g(z)$ is the inverse of the indefinite integral of $p(x)$. □

The **inverse transform sampling** is the most basic sampling method.

- It is useful for distributions whose inverse of CDF is easy to obtain.
- In other word, in is impossible to use this method if the dimension is too high or the inverse of CDF is impossible to calculate.

Here is a specific method for generating samples from a two-dimensional Gaussian distribution, which is called the **Box-Muller method**.

Algorithm 2: Box-Muller algorithm

Input : empty**Output:** $y_1, y_2 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{2 \times 2})$

```
1  $z_1 \sim \mathcal{U}(-1, 1)$ 
2  $z_2 \sim \mathcal{U}(-1, 1)$ 
3  $r \leftarrow z_1^2 + z_2^2$ 
4 repeat
5   if  $z_1^2 + z_2^2 \leq 1$  then
6      $y_1 \leftarrow z_1 \left( \frac{-2 \log r^2}{r^2} \right)^{\frac{1}{2}}$ 
7      $y_2 \leftarrow z_2 \left( \frac{-2 \log r^2}{r^2} \right)^{\frac{1}{2}}$ 
8   else
9      $\perp$  discard samples
10 until The samples are not discarded
11 return  $y_1, y_2$ 
```

Proof. Try to calculate the joint distribution of y_1 and y_2 . □

We can obtain the samples from Gaussian distribution with mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$ (Cholesky decomposition), by transforming the samples through linear transformation $\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{y}$.

2.2 Rejection Sampling

The **rejection sampling** allow us to pick samples from complex distribution $p(z)$. We use proposal function $q(z)$ which is easier to sample from. Furthermore, we suppose the value of $p(z)$ is easy to calculate up to normalization constant Z_p , i.e., $\tilde{p}(z)$ satisfying $p(z) = \frac{1}{Z_p} \tilde{p}(z)$ is easy to calculate.

Algorithm 3: Rejection sampling algorithm

Input : function $\tilde{p}(\cdot)$ satisfying $p(z) = \frac{1}{Z_p} \tilde{p}(z)$ where the desired distribution is $p(\cdot)$ constant k proposal distribution $q(\cdot)$ satisfying $kq(z) \geq \tilde{p}(z)$ **Output:** $z \sim p(z)$

```
1 repeat
2    $x \sim q(z)$ 
3    $u \sim \mathcal{U}(0, kq(x))$ 
4   if  $u \leq \tilde{p}(z)$  then
5     accept sample  $u$ 
6   else
7      $\perp$  reject sample  $u$ 
8 until The sample is accepted
9 return  $u$ 
```

Proof. Let $X \sim q(z)$ and let A be the event of X being accepted. What we have to prove is $P(X \leq x|A) = \int_{-\infty}^x p(t)dt$.

Since $P(X \leq x, A) = \int_{-\infty}^x P(A|X=t)q(t)dt$, and $P(A|X=t) = \frac{\tilde{p}(t)}{kq(t)}$, $P(X \leq x, A) = \frac{1}{k} \int_{-\infty}^x \tilde{p}(t)dt$.

And $P(A) = P(X \leq \infty, A) = \frac{1}{k} \int_{-\infty}^{\infty} \tilde{p}(t)dt = \frac{Z_p}{k}$. Then

$$\begin{aligned} P(X \leq x|A) &= \frac{P(X \leq x, A)}{P(A)} = \frac{\frac{1}{k} \int_{-\infty}^x \tilde{p}(t)dt}{\frac{Z_p}{k}} \\ &= \int_{-\infty}^x \frac{\tilde{p}(t)}{Z_p} dt = \int_{-\infty}^x p(t)dt \end{aligned}$$

□

The constant k must be as small as possible subject to the constraint $kq(z) \geq \tilde{p}(z)$, to reduce rejection counts.

- We can sample from complex distributions such as gamma distribution, using the rejection sampling;
- If the dimension of the distribution gets higher, the number of rejected samples increase tremendously, leading to inefficiency.

2.3 Adaptive Rejection Sampling

In rejection sampling, rejected samples are discarded. In **adaptive rejection sampling**, we don't just discard the rejected samples. We use it to improve the envelope function which is used as proposal distribution. The construction of envelope function is straightforward if the desired distribution $p(x)$ is log-concave, i.e., $\log p(z)$ has non-increasing derivatives. Even if $p(x)$ is not log-concave, we still can use adaptive sampling if we follow each rejection sampling step with a Metropolis-Hastings step.

Algorithm 4: Adaptive rejection sampling algorithm

Input : log-concave distribution $p(\cdot)$
 set of grid points $Z = \{z_1, z_2, \dots, z_N\}$
 constant k

Output: $z \sim p(z)$

```

1 repeat
2   Construct the envelope function  $\log q(x)$  of  $\log p(x)$  over grid points  $Z$ 
3    $z \sim q(x)$ 
4    $u \sim \mathcal{U}(0, kq(z))$ 
5   if  $u \leq p(x)$  then
6     accept sample  $u$ 
7   else
8      $Z \leftarrow Z \cup \{z\}$ 
9     reject sample  $u$ 
10 until The sample is accepted
11 return  $u$ 

```

2.4 Importance Sampling

Importance sampling is precisely not a sampling method. It is used to calculate expectation of given random variable. As with rejection sampling, importance sampling uses proposal distribution for calculating expectation of the function of random variable following desired distribution. The importance sampling algorithm is mainly used in off-policy reinforcement learning.

Algorithm 5: Importance sampling algorithm

Input : probability distribution $p(\cdot)$
 proposal distribution $q(\cdot)$
 function $f(\cdot)$

Output: $\mathbb{E}_{x \sim p(x)}[f(x)]$

```
1  $\{z_1, z_2, \dots, z_N\} \sim q(x)$ 
2  $w_i \leftarrow \frac{p(z_i)}{q(z_i)}$  for  $i = 1, 2, \dots, N$ 
3  $E \leftarrow \frac{1}{N} \sum_{i=1}^N f(z_i)w_i$ 
4 return  $E$ 
```

Proof. The main idea is to manipulate the equation $\mathbb{E}[f(z)] = \int f(z)p(z)dz$ to $\int f(z)\frac{p(z)}{q(z)}q(z)dz$. □

The quantities $w_i = \frac{p(z_i)}{q(z_i)}$ are called **importance weights**, and they compensate for the samples which are sampled more or less compared to the desired distribution. Importance sampling uses the **Monte Carlo** method, which uses sampling to integrate complicated integrands.

- If we construct $q(z)$ very well according to the target space, we can significantly decrease the variance of expectation much better than ordinary Monte Carlo method.
- However, if we construct $q(z)$ so that it is small at regions where $p(z)f(z)$ is very large, the variance may be small even if the estimate of the expectation may be severely wrong. Moreover, it is hard to construct $q(z)$ if the dimension is very high.

2.5 Sampling-importance-resampling

Sampling-importance-resampling method revises importance sampling method so that it can actually create samples.

Algorithm 6: Importance sampling algorithm

Input : desired distribution $p(\cdot)$
 proposal distribution $q(\cdot)$

Output: $z \sim p(z)$

```
1  $\{z_1, z_2, \dots, z_N\} \sim q(z)$ 
2  $w_i \leftarrow \frac{p(z_i)}{q(z_i)}$  for  $i = 1, 2, \dots, N$ 
3  $w_i \leftarrow \frac{w_i}{\sum_{j=1}^N w_j}$  for  $i = 1, 2, \dots, N$ 
4  $r \sim \{z_1, z_2, \dots, z_N\}$ , following  $\mathcal{M}(w_1, w_2, \dots, w_N)$ 
5 return  $r$ 
```

Proof. We can prove this by showing that for any test function $g(x)$, $\mathbb{E}[g(T)] = \int g(t)p(t)dt$, where T is the resampled target variable.

Let $\{z_1, z_2, \dots, z_N\}$ be the N initial samples drawn i.i.d. from the proposal distribution $q(z)$. The target variable T is obtained by resampling from this set, meaning T takes the value of z_i with the normalized importance weight w_i :

$$\mathbb{P}(T = z_i \mid z_1, z_2, \dots, z_N) = w_i = \frac{p(z_i)/q(z_i)}{\sum_{j=1}^N p(z_j)/q(z_j)}.$$

Using the law of iterated expectation by conditioning on the initial samples $\mathbf{z} = \{z_1, \dots, z_N\}$, we can expand $\mathbb{E}[g(T)]$ as follows:

$$\begin{aligned} \mathbb{E}[g(T)] &= \mathbb{E}_{z_1, \dots, z_N \sim q} [\mathbb{E}_{T \sim \mathcal{M}(w_1, \dots, w_N)} [g(T) \mid z_1, z_2, \dots, z_N]] \\ &= \mathbb{E}_{z_1, \dots, z_N \sim q} \left[\sum_{i=1}^N w_i g(z_i) \right] \quad (\text{since } T \text{ becomes } z_i \text{ with prob } w_i) \\ &= \mathbb{E}_{z_1, \dots, z_N \sim q} \left[\frac{\frac{1}{N} \sum_{i=1}^N \frac{p(z_i)}{q(z_i)} g(z_i)}{\frac{1}{N} \sum_{j=1}^N \frac{p(z_j)}{q(z_j)}} \right] \end{aligned}$$

By the law of large numbers, as $N \rightarrow \infty$, the sample means of the initial particles $z_{1:N}$ converge non-randomly to their expectations under $q(z)$:

$$\begin{aligned} \frac{1}{N} \sum_{i=1}^N \frac{p(z_i)}{q(z_i)} g(z_i) &\longrightarrow \mathbb{E}_{z \sim q} \left[\frac{p(z)}{q(z)} g(z) \right] = \int \frac{p(z)}{q(z)} g(z) q(z) dz = \int p(z) g(z) dz \\ \frac{1}{N} \sum_{j=1}^N \frac{p(z_j)}{q(z_j)} &\longrightarrow \mathbb{E}_{z \sim q} \left[\frac{p(z)}{q(z)} \right] = \int \frac{p(z)}{q(z)} q(z) dz = 1 \end{aligned}$$

Since the terms inside the expectation converge to deterministic constants, the outer expectation with respect to the initial samples $z_{1:N}$ resolves, and we get:

$$\mathbb{E}[g(T)] = \mathbb{E}_{z_{1:N} \sim q} \left[\frac{\int p(z) g(z) dz}{1} \right] = \int p(t) g(t) dt.$$

□

- We don't have to take care of constant k and normalization constant Z_p when using proposal distribution,
- We can increase the effective sample size, by discarding meaningless samples.
- However, since the samples with low probability are kept discarded, the diversity of samples decreases.

3 Markov Chain Monte Carlo

3.1 Introduction

The main idea of Markov chain Monte Carlo is to use the *stationary distribution* of *Markov chain*. If we can construct Markov chain with our desired distribution $p(x)$ as stationary distribution, we will be able to collect the sample from $p(x)$ easily by just following the chain.

3.2 Markov Chain

The **Markov chain** is defined to be a series of random variables such that each random variable only depends on its previous variable.

Definition 3.2.1 (Markov Chain)

The **Markov chain** is a sequence of random variables $\{z_1, z_2, \dots, z_N\}$ satisfying following conditional independence property for $n \in \{1, \dots, N\}$;

$$p(z_{i+1}|z_i, z_{i-1}, \dots, z_1) = p(z_{i+1}|z_i)$$

The **transition probabilities** $T_n(z_n, z_{n+1}) = p(z_{n+1}|z_n)$ is used to specify the Markov chain. If the transition probabilities are same for all n , then the Markov chain is called *homogeneous*.

Definition 3.2.2 (Stationary Distribution with respect to a Homogeneous Markov Chain)

The probability distribution $p(z)$ is said to be **stationary with respect to a homogeneous Markov chain** if $p(z)$ satisfies following condition for a transition probability T :

$$p(z) = \int T(z', z)p(z')dz'$$

for all z in Markov chain.

As the Markov chains gets longer so that the sequence follows stationary distribution, the random variable afterward will follow the stationary distribution, too. So if we can construct a Markov chain with the specific stationary distribution, we can get samples from that stationary distribution.

The following condition is a sufficient but not necessary condition for ensuring that the distribution $p(z)$ is stationary about Markov chain.

Theorem 3.2.1 (Detailed Balance)

Let $p(z)$ be a probability distribution and let T be a transition probability of a homogeneous Markov chain. Then, $p(z)$ is stationary with respect to the Markov chain if it satisfies the **detailed balance** condition:

$$p(z)T(z, z') = p(z')T(z', z)$$

for all pairs of z and z' of Markov chain.

3.3 Metropolis-Hastings Algorithm

Algorithm 7: Metropolis-Hastings algorithm

Input : unnormalized distribution $\tilde{p}(z)$
proposal distributions $\{q_k(z|\hat{z}) : k \in 1, \dots, K\}$
mapping from iteration index to distribution index $M(\cdot)$
initial state $z^{(0)}$
number of iterations T

Output: $z \sim p(z)$

```

1  $z_{prev} \leftarrow z^{(0)}$ 
2 for  $\tau \in 1, \dots, T$  do
3    $k \leftarrow M(\tau)$ 
4    $z' \sim q_k(z|z_{prev})$ 
5    $u \sim \mathcal{U}(0, 1)$ 
6   if  $\frac{\tilde{p}(z')q(z_{prev}|z')}{\tilde{p}(z_{prev})q(z'|z_{prev})} > u$  then
7      $z_{prev} \leftarrow z'$ 
8   else
9      $z_{prev} \leftarrow z_{prev}$ 
10 return  $z_{prev}$ 

```

The Metropolis-Hastings algorithm at a specific step τ draws a sample from proposal distribution and accept it with probability $A_k(z', z^{(\tau)})$ where

$$A_k(z', z^{(\tau)}) = \min \left(1, \frac{\tilde{p}(z')q_k(z^{(\tau)}|z')}{\tilde{p}(z^{(\tau)})q_k(z'|z^{(\tau)})} \right).$$

Proof. We show that the distribution $p(z)$ is stationary distribution of Markov chain with following transition probabilities; $T_k(z, z') = q_k(z'|z)A_k(z, z')$. This is easily proved by using Theorem 3.2.1. $\square$

Since the samples that are within small steps are highly correlated, we collect samples at each 10 or more steps. This is called *thinning*. In the high-dimensional spaces, the accept probability of samples gets very low and meaningful samples are difficult to obtain. To overcome this, we use **Gibbs Sampling**.

3.4 Gibbs Sampling

The **Gibbs sampling** is a kind of Markov chain Monte Carlo algorithm, and can be seen as a special case of Metropolis-Hastings algorithm.

Algorithm 8: Gibbs sampling algorithm

Input : initial values $\{z_1, \dots, z_M\}$
conditional distributions $\{p(z_i | \{z_{j \neq i}\}) : i \in \{1, \dots, M\}\}$
number of iterations T
Output: final values $\{z_1, \dots, z_M\}$

```
1 for  $\tau \in \{1, \dots, T\}$  do
2   for  $i \in \{1, \dots, M\}$  do
3      $z_i \sim p(z_i | \{z_{j \neq i}\})$ 
4 return  $\{z_1, \dots, z_M\}$ 
```

TODO:Gibbs sampling can be obtained from Metropolis-Hastings algorithm;

3.5 Ancestral Sampling

Ancestral sampling algorithm is mainly used for probabilistic graphical models. When the joint distribution is given by

$$p(\mathbf{z}) = \prod_{i=1}^M p(\mathbf{z}_i | \text{pa}(i))$$

we traverse the nodes in its topological order, picking a sample from distribution $p(\mathbf{z}_i | \text{pa}(i))$ at each variable node. This is possible, since traversing in topological order ensures that all the parent nodes are visited before visiting certain node.

Algorithm 9: Ancestral sampling algorithm

Input : joint distribution $p(\mathbf{z}) = \prod_{i=1}^M p(\mathbf{z}_i | \text{pa}(i))$ where nodes are given in topolgal order $\{\mathbf{z}_1, \dots, \mathbf{z}_M\}$
Output: $\mathbf{z} \sim p(\mathbf{z})$

```
1 for  $i \in \{1, \dots, M\}$  do
2    $\mathbf{z}_i^* \sim p(\mathbf{z}_i | \text{pa}(i))$ 
3 return  $\mathbf{z}^*$ 
```

TODO:likelihood weight sampling

4 Langevin Sampling